BROADWAY INFOSYS
Since 2008

CERTIFIED ISO 9001:2015 COMPANY

Building Global IT Professionals since 2008

15K+
Certified Students

152+
Courses

70+
Qualified Teachers

www.broadwayinfosys.com



Python Programming

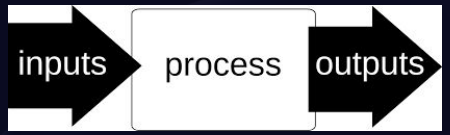
From Zero to Programming Hero — Your First Step into Programming

Shristi Shrestha

Broadway Infosys - Python Django Trainer
Leapfrog Technology - FullStack Developer

What is a program?

1. A program is a set of instructions that tells a computer what to do.



2. It guides the computer through specific tasks or problems using a programming language.



3. The program helps to automate tasks, process data, create interfaces, and perform calculations based on the given instructions.

What is a program? (Contd.)

4. Types of Programming Languages:
    a. Low-Level Language: Language understood by machines.
        - Machine Language: Written in 0's and 1's
        - Assembly Language: Written in English words and requires an assembler to translate the words into machine language.
    b. High-Level Language: Language understood by humans.
        - C++, Java, Python, etc.

Python's Roots and Its Ecosystem

Understanding the Origin, Growth, and Vast Reach of Python

What is Python?

- High-level, interpreted programming language
- Designed for readability and simplicity
- Created by Guido van Rossum and released in 1991
- Supports multiple programming paradigms (OOP, procedural, functional)

History and Roots

- Developed in the late 1980s as a successor to ABC language
- Officially released in 1991
- Named after 'Monty Python’s Flying Circus', not the snake
- Emphasis on code readability and developer productivity

Python Philosophy

'The Zen of Python' by Tim Peters

- Key principles:
  - Beautiful is better than ugly
  - Simple is better than complex
  - Readability counts

Python Ecosystem by Domain

- Web Development – Django, Flask, FastAPI
- Data Science – NumPy, pandas, Matplotlib
- Machine Learning – scikit-learn, TensorFlow, PyTorch
- Automation/Scripting – Selenium, Requests
- DevOps – Ansible, Fabric
- Game Development – Pygame
- Desktop Apps – Tkinter, PyQt

Why Python Dominates Today

- Versatility across domains
- Large community support
- Rich ecosystem and package availability
- Simple syntax for beginners and pros alike

The Python Programming Language

Python is a high-level programming language known for its simplicity, readability, and versatility. Here are some key features of Python:

- **Interpretive nature:** Python is an interpreted language, which means that code can be executed directly without the need for compilation.
- **Easy to learn and use:** Python has a clean and intuitive syntax, making it easy for beginners to understand and write code.
- **Readability:** Python code is highly readable due to its emphasis on indentation and a clear, expressive syntax.
- **Large and active community:** Python has a large and supportive community of developers who contribute to its growth and share knowledge.
- **Cross-platform compatibility:** Python is available on various platforms, including Windows, macOS, Linux, and more.
- **Extensive standard library**: Python provides a comprehensive standard library that includes modules for performing various tasks, such as working with files, networking, database interaction, and more.
- **Third-party libraries and frameworks:** Python boasts a vast ecosystem of third-party libraries and frameworks, such as NumPy, Pandas, TensorFlow, Django, Flask, and many more.

Download Python

Download latest version of Python

from official website : www.python.org

Goto:

https://www.python.org/downloads/




```
C:\Users\Your Name>python --version
```



Python Package Index (PyPI)

- Repository of over 400,000 packages
- Used to share and install Python libraries
- Tools: pip, poetry, conda
- Community-driven contributions

PIP

- pip is the package installer for Python.
- It allows you to install, manage, and uninstall Python packages from the Python Package Index (PyPI) and other package indexes.
- It's an essential tool for Python developers, enabling the use of third-party libraries and dependencies that are not part of the Python standard library.



Download IDE

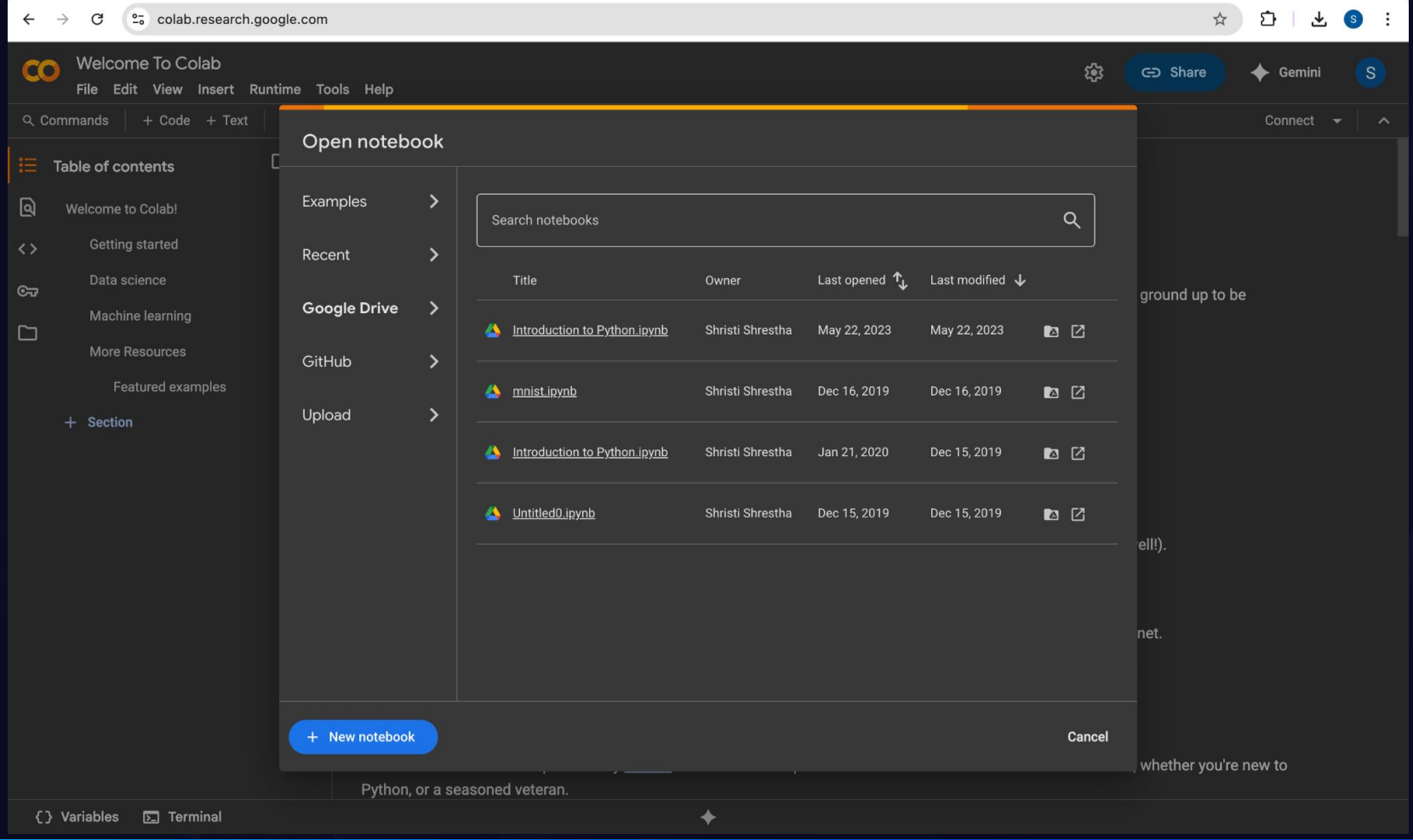
Choose an IDE for a user-friendly development experience. Popular options include PyCharm, VS Code, and JupyterLab.

Jupyterlab: pip install jupyterlab

To run jupyterlab: jupyter-lab or jupyter lab

VS Code: https://code.visualstudio.com/

Pycharm: https://www.jetbrains.com/pycharm/download/


colab.research.google.com
Welcome To Colab
File Edit View Insert Runtime Tools Help
Share
Gemini
Commands
+ Code
+ Text
Connect
Table of contents
Welcome to Colab!
Getting started
Data science
Machine learning
More Resources
Featured examples
+ Section
Open notebook
Examples
Recent
Google Drive
GitHub
Upload
Search notebooks
Title
Owner
Last opened
Last modified
Introduction to Python.ipynb
Shristi Shrestha
May 22, 2023
May 22, 2023
mnist.ipynb
Shristi Shrestha
Dec 16, 2019
Dec 16, 2019
Introduction to Python.ipynb
Shristi Shrestha
Jan 21, 2020
Dec 15, 2019
Untitled0.ipynb
Shristi Shrestha
Dec 15, 2019
Dec 15, 2019
+ New notebook
Cancel
ground up to be
ell!).
net.
whether you're new to
Python, or a seasoned veteran.
Variables
Terminal

Values and Types

- In programming, a value is a piece of data that can be stored or manipulated.
- Every value has a type, which defines the kind of data it represents, such as numbers, strings, or booleans.
- Understanding values and types helps in working with different data in your program.
    - # Integer value (type: int)

      age = 25
    - # String value (type: str)

      name = "John Doe"
    - # Boolean value (type: bool)

      is_student = True

Variables

1. A **variable** is a named place in the memory where a programmer can store data and later retrieve the data using the **variable** “name”.
2. Programmers get to choose the names of the **variables.**
3. You can change the contents of a variable in a later statement.

x = 10.2
y = 22

x	10.2
y	22

Variable Basics

Variables are created by assigning values to them using the assignment operator (=). They can have different data types such as integers, floats, strings, and booleans.

```
# Variable declaration and assignment
name = "John Doe"
age = 25
is_student = True
```

Variable Naming

1. A variable name must start with a letter or the underscore character
2. A variable name cannot start with a number
3. A variable name can only contain alphanumeric characters and underscores (A-z, 0-9, and _)
4. Variable names are case-sensitive (age, Age and AGE are three different variables)
5. A variable name cannot be any of the Python keywords.

Values and Types (Contd.)

1. Numeric Types:
    - Integers (int): Whole numbers without decimal points, such as 10, -5, or 0.
    - Floating-Point Numbers (float): Numbers with decimal points or in scientific notation, such as 3.14, -0.5, or 2e3 (2000).
    - Complex (complex): Imaginary number such as 1j
2. String Type:
    - Strings (str): Ordered sequences of characters enclosed in single (' ') or double (" ") quotes, such as "Hello" or 'Python'.
3. Boolean Type:
    - Boolean (bool): Represents the truth value of an expression, either True or False. Booleans are fundamental for logical operations and conditional statements.

Values and Types (Contd.)

None Type:

- None: A special value in Python that represents the absence of a value or the absence of a return value from a function. It is often used as an initial or default value.

Immutable vs. Mutable Types:

- Immutable Types: Some types, like numbers and strings, are immutable, meaning their values cannot be changed once assigned.
- Mutable Types: Other types, like lists and dictionaries, are mutable, allowing modifications to their contents.

Type Checking

The type() function returns the type of a value. It helps in understanding the type of a variable or expression and can be used for conditional checks.

```
x = 5
y = 3.14
z = "Hello, World!"

print(type(x))  # Output: <class 'int'>
print(type(y))  # Output: <class 'float'>
print(type(z))  # Output: <class 'str'>
```

Type Conversion

Conversion Functions: Convert values between different types using built-in conversion functions like int(), float(), str(), and bool().

```
# Conversion to integer (int)
number_str = "10"
number = int(number_str)
print(number)
print(type(number))

# Conversion to float
pi_str = "3.14"
pi = float(pi_str)
print(pi)
print(type(pi))
```

Data Types and Type Inference

Python is a dynamically typed language, meaning the type of a variable is inferred based on the assigned value.

```
# Examples of variables with different data types
x = 10
pi = 3.14
message = "Hello, world!"
is_valid = False
```

Hello World

Create a new file main.py and in the terminal run

> python main.py

Go to terminal and in the terminal run

> python

> print(‘hello world’)

Multiple Assignments and Unpacking

Python allows multiple variables to be assigned values simultaneously and supports unpacking, where multiple values can be assigned to multiple variables at once.

```
# Multiple assignments
a, b, c = 1, 2, 3

# Unpacking
name, age, city = ("Alice", 30, "New York")
```

Keywords

Keywords are reserved words with specific meanings in Python.

Python has several keywords with special meanings used for control flow, defining functions, loops, and more.

```
False               break               for                 not
None                class               from                or
True                continue            global              pass
__peg_parser__      def                 if                  raise
and                 del                 import              return
as                  elif                in                  try
assert              else                is                  while
async               except              lambda              with
await               finally             nonlocal            yield
```

Best Practices

1. Choose meaningful variable names that reflect their purpose.
2. Use lowercase letters and underscores for variable names (snake_case).
3. Avoid using reserved keywords as variable names.
4. Be mindful of variable scoping and avoid naming conflicts.

Input and Output Functions

print() function for output.

input() function for input. Input is always string by default.

Working with `format()`, f-strings, & Escape Sequences

Concepts to Cover:

- `format()` method for string formatting.
- f-strings (Python 3.6+): Easier and cleaner.
- Escape sequences:
  - `\n` - Newline
  - `\t` - Tab
  - `\\` - Backslash
  - `\"` or `\'` - Quotes

```python
# Using format()
print("My name is {} and I am {} years old.".format(name, age))

# Using f-string
print(f"My name is {name} and I am {age} years old.")

# Escape sequences
print("Hello,\nWorld!")   # Newline
print("Name:\tShristi")   # Tab
print("He said, \"Python is fun!\"")  # Quotes
```

Operators and Operands

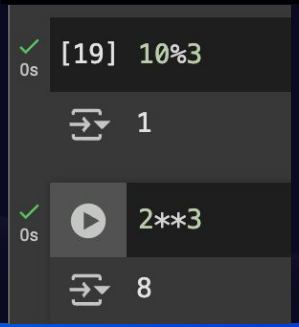
In Python, operators are special symbols or keywords that perform operations on operands.

Operands, on the other hand, are the values or variables that the operators act upon.

Arithmetic Operators:

- Addition (+): Adds two operands together.
- Subtraction (-): Subtracts the second operand from the first.
- Multiplication (*): Multiplies two operands.
- Division (/): Divides the first operand by the second.
- Modulus (%): Returns the remainder of division.
- Exponentiation (**): Raises the first operand to the power of the second.

```
# Arithmetic operators
x = 10
y = 5
sum_result = x + y
difference_result = x - y
product_result = x * y
division_result = x / y
```



Comparison Operators:

- Equal to (==): Checks if two operands are equal.
- Not equal to (!=): Checks if two operands are not equal.
- Greater than (>): Checks if the first operand is greater than the second.
- Less than (<): Checks if the first operand is less than the second.
- Greater than or equal to (>=): Checks if the first operand is greater than or equal to the second.
- Less than or equal to (<=): Checks if the first operand is less than or equal to the second.

```
# Comparison operators
is_equal = x == y
is_greater = x > y
is_less_or_equal = x <= y
```

Assignment Operators:

- Assignment (=): Assigns the value on the right to the variable on the left.
- Compound assignment operators (+=, -=, *=, /=, etc.): Perform an operation and assign the result to the variable.

Logical Operators:

- Logical AND (and): Returns True if both operands are True.
- Logical OR (or): Returns True if at least one operand is True.
- Logical NOT (not): Returns the opposite of the operand's value.

```
# Logical operators
is_true = True
is_false = False
logical_and = is_true and is_false
logical_or = is_true or is_false
logical_not = not is_true
```

THANK YOU!

BROADWAY INFOSYS
Since 2008

Shree Ganesh Marg, Subidhanagar, Tinkune, Kathmandu
01-4117578 / 4111849 / 984100200
www.broadwayinfosys.com